

UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACION ORIENTADA A OBJETOS

DIEGO ISAIAS MARROQUIN MOSCOSO

EJERCICIO #2

Análisis

Responda las siguientes preguntas:

1. ¿Qué propiedades y métodos tendrá cada clase?

La clase Jugador tendrá como propiedades nombre, nickname, edad, puntajes y partidasJugadas. Sus métodos serán: el constructor Jugador(nombre, nickname, edad); los getters getNombre(), getNickname(), getEdad(), getPartidasJugadas() y getPartidasDisponibles(); los setters setNombre(), setNickname() y setEdad(); y los métodos de comportamiento registrarPartida(), consultarPuntaje(), modificarPuntaje(), obtenerPuntajes(), calcularPromedio(), obtenerPuntajeMayor() y obtenerPuntajeMenor().

La clase Main no requiere propiedades; únicamente contendrá el método estático main(), encargado de todo el flujo del programa: crear al jugador, mostrar el menú de nueve opciones y comunicarse con la clase Jugador.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

En Jugador: nombre y nickname deben ser String porque almacenan texto; edad y partidasJugadas deben ser int porque guardan números enteros; puntajes debe ser un arreglo de enteros (int[]) porque guarda un puntaje por cada partida.

Entre los métodos: registrarPartida() y modificarPuntaje() devuelven boolean porque informan si la operación fue posible; consultarPuntaje(), getEdad(), getPartidasJugadas(),

getPartidasDisponibles(), obtenerPuntajeMayor() y obtenerPuntajeMenor() devuelven int; calcularPromedio() devuelve double porque el promedio generalmente produce decimales; obtenerPuntajes() devuelve int[]; los setters devuelven void. En Main, main() es void y recibe String[] args.

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo? ¿Qué tipo de datos almacenará?

La propiedad puntajes es la que debe implementarse con un arreglo, declarado como int[10], ya que la guía pide guardar como máximo diez partidas por jugador y es necesario conservar el puntaje individual de cada una para poder consultarlo, modificarlo y calcular estadísticas. Almacena datos de tipo entero (int), con valores válidos entre 0 y 100.

4. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

En la clase Jugador todos los atributos deben ser private para aplicar el encapsulamiento y proteger el estado interno del objeto; el constructor, los getters, los setters y los métodos de comportamiento deben ser public para que Main pueda utilizarlos. En la clase Main el método main debe ser public static void, como lo exige Java como punto de entrada del programa; esa clase no declara atributos.

5. ¿Qué parámetros serán requeridos por los métodos en sus clases?

- Constructor Jugador(nombre: String, nickname: String, edad: int).
- setNombre(nombre: String), setNickname(nickname: String) y setEdad(edad: int): reciben el nuevo valor a asignar.
- registrarPartida(puntaje: int): recibe el puntaje de la nueva partida.
- consultarPuntaje(partida: int): recibe el número de la partida a consultar.
- modificarPuntaje(partida: int, puntaje: int): recibe la partida a editar y su nuevo puntaje.

- `getNombre()`, `getNickname()`, `getEdad()`, `getPartidasJugadas()`, `getPartidasDisponibles()`, `obtenerPuntajes()`, `calcularPromedio()`, `obtenerPuntajeMayor()` y `obtenerPuntajeMenor()`: no requieren parámetros.
- `main(args: String[])`: recibe el arreglo de argumentos estándar de Java.

6. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Mediante el constructor de `Jugador`, que recibe nombre, nickname y edad y los asigna a sus respectivos atributos. Además crea el arreglo `puntajes` con `new int[10]`, quedando sus diez posiciones inicializadas en cero por defecto, y coloca `partidasJugadas` en cero porque al momento de crear el objeto todavía no existe ninguna partida registrada.

7. ¿Cómo determinará cuál es la siguiente posición disponible dentro del arreglo?

El contador `partidasJugadas` cumple ese papel: siempre coincide con la primera posición libre del arreglo. Al registrar una partida se guarda el puntaje en `puntajes[partidasJugadas]` y después se incrementa el contador. Antes de hacerlo se comprueba que `partidasJugadas` sea menor a 10 para no salir de los límites del arreglo.

8. ¿Cómo recorrerá únicamente las posiciones del arreglo que contienen puntajes registrados?

Todos los recorridos utilizan como límite el contador `partidasJugadas` y no la longitud total del arreglo: los ciclos inician en $i = 0$ y continúan mientras $i < \text{partidasJugadas}$. De esta forma solo se visitan las posiciones con puntajes realmente registrados, ignorando las que permanecen en cero. Adicionalmente, `obtenerPuntajes()` copia únicamente esas posiciones a un arreglo auxiliar del tamaño exacto antes de devolverlo a `Main`.

Requisitos funcionales del sistema:

- El sistema debe dar la bienvenida y solicitar al usuario su nombre, edad y nickname para crear al jugador activo.
- El sistema debe permitir crear un nuevo jugador desde el menú en cualquier momento, reemplazando al jugador activo anterior.
- Cada jugador puede registrar como máximo diez partidas.
- El sistema debe permitir registrar el puntaje de una partida validando que esté entre 0 y 100 puntos.
- El sistema debe rechazar registros con puntajes fuera del rango permitido o cuando ya existan diez partidas registradas.
- El sistema debe permitir consultar todos los puntajes registrados mostrando el número de cada partida.
- El sistema debe permitir consultar el puntaje de una partida específica e informar cuando la partida elegida no existe o no se ha jugado.
- El sistema debe permitir modificar el puntaje de una partida ya jugada, validando que la partida exista y que el nuevo puntaje esté entre 0 y 100.
- El sistema debe calcular y mostrar el promedio de los puntajes registrados del jugador activo.
- El sistema debe determinar y mostrar el puntaje mayor y el puntaje menor entre todas las partidas registradas.
- El sistema debe informar cuántas partidas se han registrado y cuántas partidas siguen disponibles.
- El menú principal debe repetirse hasta que el usuario elija salir y debe advertir cuando se ingresa una opción inválida.

Jugador

- nombre: String
- nickname: String
- edad: int
- puntajes: int[]
- partidasJugadas: int

- + Jugador(nombre: String, nickname: String, edad: int)
- + getNombre(): String
- + getNickname(): String
- + getEdad(): int
- + getPartidasJugadas(): int
- + getPartidasDisponibles(): int
- + setNombre(nombre: String): void
- + setNickname(nickname: String): void
- + setEdad(edad: int): void
- + registrarPartida(puntaje: int): boolean
- + consultarPuntaje(partida: int): int
- + modificarPuntaje(partida: int, puntaje: int): boolean
- + obtenerPuntajes(): int[]
- + calcularPromedio(): double
- + obtenerPuntajeMayor(): int
- + obtenerPuntajeMenor(): int

Propósito de la clase: Esta clase representa al jugador inscrito en el torneo universitario de videojuegos. Almacena sus datos personales (nombre, nickname y edad) y administra el registro de sus puntajes por partida mediante un arreglo de máximo diez posiciones junto con un contador de partidas jugadas. Concentra toda la lógica del dominio: registrar nuevas partidas, consultar y modificar puntajes, entregar el historial completo y calcular estadísticas como el promedio, el puntaje mayor, el puntaje menor y las partidas disponibles.

Propósito de los atributos:

- nombre: almacena el nombre real del jugador ingresado en consola. Permite identificarlo dentro del torneo y mostrarlo en los mensajes del programa.
- nickname: almacena el apodo con el que el jugador participa. Sirve como identificador dentro del juego y complementa los datos personales mostrados.
- edad: almacena la edad del jugador, dato solicitado al momento de crearlo y necesario para el registro del participante en el torneo.
- puntajes: arreglo de enteros de diez posiciones que almacena el puntaje obtenido en cada partida jugada (valores entre 0 y 100). Es el atributo identificado para implementarse con un arreglo, ya que conserva el historial completo de puntajes necesario para las consultas, modificaciones y estadísticas.
- partidasJugadas: contador que indica cuántas posiciones del arreglo están en uso. Controla el límite de diez partidas, marca la siguiente posición disponible y delimita todos los recorridos sobre el arreglo.

Propósito de los métodos:

Constructor Jugador(nombre, nickname, edad): recibe los tres datos personales del jugador y los asigna a sus atributos; además inicializa puntajes como un arreglo de diez enteros en cero y partidasJugadas en cero, dejando el objeto listo para utilizarse.

getNombre(): no requiere parámetros. Retorna el nombre almacenado para identificar al jugador o mostrarlo en pantalla.

getNickname(): no requiere parámetros. Retorna el apodo del jugador activo.

getEdad(): no requiere parámetros. Retorna la edad registrada del jugador.

getPartidasJugadas(): no requiere parámetros. Retorna cuántas partidas tienen puntaje registrado hasta el momento.

getPartidasDisponibles(): no requiere parámetros. Calcula y retorna 10 menos partidasJugadas para informar al usuario cuántas partidas aún puede jugar.

setNombre(nombre): recibe un String con el nuevo nombre y reemplaza el valor almacenado en el atributo nombre.

setNickname(nickname): recibe un String con el nuevo apodo y lo asigna al atributo nickname.

setEdad(edad): recibe un entero con la nueva edad y actualiza el valor guardado.

registrarPartida(puntaje): recibe el puntaje de la nueva partida. Valida que esté entre 0 y 100 y que exista cupo disponible (menos de diez partidas jugadas); si todo es válido lo guarda en la siguiente posición libre del arreglo, incrementa partidasJugadas y retorna true. Si alguna validación falla retorna false sin alterar ningún dato.

consultarPuntaje(partida): recibe el número de la partida que se desea conocer (numeración desde 1). Lo convierte al índice correspondiente del arreglo y verifica que esté dentro del rango ya jugado; retorna el puntaje almacenado en esa partida o -1 cuando no existe, valor que Main interpreta para mostrar un mensaje de partida inválida.

modificarPuntaje(partida, puntaje): recibe el número de la partida a editar y el nuevo puntaje. Comprueba primero que la partida ya haya sido jugada y luego que el puntaje esté entre 0 y 100; si ambas condiciones se cumplen actualiza la posición del arreglo y retorna true; en caso contrario retorna false sin realizar cambios.

obtenerPuntajes(): no requiere parámetros. Construye un arreglo auxiliar del tamaño exacto de partidasJugadas, copia en él únicamente los puntajes registrados y lo retorna para que Main pueda listarlos; retorna null si todavía no hay ninguna partida, lo que también sirve como señal de error para Main.

calcularPromedio(): no requiere parámetros. Suma los puntajes de todas las partidas jugadas y divide el total entre partidasJugadas, retornando el promedio como double; retorna -1 cuando no existen partidas registradas, indicando a Main que debe avisar al usuario.

obtenerPuntajeMayor(): no requiere parámetros. Recorre las posiciones ocupadas del arreglo comparando los valores y retorna el puntaje más alto obtenido; retorna -1 si no hay partidas registradas.

obtenerPuntajeMenor(): no requiere parámetros. Recorre las posiciones ocupadas del arreglo y retorna el puntaje más bajo obtenido; retorna -1 si no hay partidas registradas.

Main

+ main(args: String[]): void

Propósito de la clase: Esta clase contiene el punto de entrada del programa y es la única responsable de la interacción con el usuario en consola. Da la bienvenida, solicita los datos personales para crear al primer jugador e instancia el objeto Jugador, despliega el menú con las ocho operaciones y la opción de salida, lee todas las entradas mediante Scanner, delega toda la lógica del torneo a la clase Jugador y traduce sus valores de retorno (true, false, -1, null, arreglos y promedios) en los mensajes de éxito o error que el usuario observa.

Propósito de los atributos: ningún atributo.

Propósito de los métodos:

main(args): recibe el arreglo de argumentos estándar de Java. Crea el Scanner y un estado de menú, solicita nombre, edad y nickname para construir al primer jugador y entra a un ciclo que muestra el menú con nueve opciones: crear nuevo jugador, registrar nueva partida, consultar puntajes, consultar una partida, modificar una partida, mostrar promedio, ver mejor y peor puntaje, consultar partidas disponibles y salir del programa. Según la elección lee los datos necesarios, invoca los métodos del jugador activo, valida las opciones fuera del rango 1 a 9 y muestra los mensajes correspondientes; al elegir salir finaliza el ciclo, cierra el Scanner y termina el programa.